

Code & Image Scanning

Where It Runs

All scanning happens in **GitHub Actions** during CI — before the image is pushed to ECR. ArgoCD does not scan.

```
Push → GH Actions
├─ 1. gitleaks (secret leak check)
├─ 2. npm audit / pip-audit (dependency CVEs)
├─ 3. Bandit / Semgrep (code security)
├─ 4. Build Docker image
├─ 5. Trivy scan (image CVEs)
└─ 6. Push to ECR (only if all pass)
```

1. Secret Leak Detection

gitleaks

Scans every commit and the entire Git history for accidentally committed secrets.

What it finds: API keys, tokens, passwords, private keys, AWS credentials.

Why it matters: Once a secret is in Git history, it's exposed forever. gitleaks catches it before it reaches the repo.

Example config (`.gitleaks.toml`):

```
title = "gitleaks config"

[[rules]]
id = "aws-access-key"
description = "AWS Access Key"
regex = '''AKIA[0-9A-Z]{16}'''
tags = ["aws", "key"]
```

2. Dependency Scanning

Python — pip-audit

Checks `requirements.txt` against the OSV (Open Source Vulnerabilities) database.

What it finds: Known CVEs in installed Python packages.

CI step:

```
- name: Install deps
  run: pip install -r requirements.txt
- name: Scan dependencies
  run: pip-audit --format json --output audit-results.json
```

Node.js — npm audit

Built into npm. Checks `package-lock.json` against the npm advisory database.

What it finds: Known CVEs in npm packages and their transitive dependencies.

CI step:

```
- name: Install deps
  run: npm ci
- name: Scan dependencies
  run: npm audit --audit-level=high
```

3. Code Security Analysis

Bandit (Python)

Static analysis tool that finds common security issues in Python code.

What it finds:

- Hardcoded passwords and secrets
- SQL injection (`f"SELECT * FROM {user_input}"`)
- Unsafe `eval()` and `exec()` calls
- Weak cryptographic functions (MD5, SHA1)
- Insecure SSL verification (`verify=False`)

CI step:

```
- name: Run Bandit
  run: bandit -r apps/ -f json -o bandit-results.json --severity-level medium
```

Semgrep (All languages)

Rule-based static analysis. Works for Python, Node.js, Go, Java, and more.

What it finds: Logic-level bugs that dependency scanners miss — custom security rules, OWASP Top 10 patterns.

CI step:

```
- name: Run Semgrep
  uses: semgrep/semgrep-action@v1
  with:
    config: p/default
```

4. Docker Image Scanning

Trivy (Aqua Security)

Scans the built Docker image for vulnerabilities in OS packages, application dependencies, and misconfigurations.

What it finds:

- OS-level CVEs (Ubuntu, Alpine packages)
- Application dependency CVEs (Python pip, npm, Go modules)
- Dockerfile best practice violations (running as root, no healthcheck)
- Exposed secrets in image layers

CI step:

```
- name: Scan Docker image
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ env.ECR_URI }:${ github.sha }
    exit-code: 1
    severity: CRITICAL,HIGH
    format: table
```

Severity levels: | Level | Action | |-----|-----| | CRITICAL | Block pipeline immediately | | HIGH | Block pipeline (or warn, depending on policy) | | MEDIUM | Allow, but log for review | | LOW | Ignore |

5. Pipeline Failure Behavior

If any scanner finds a CRITICAL or HIGH vulnerability:

1. **Pipeline fails** — the image is NOT pushed to ECR
2. **Results are saved** as artifacts for review
3. **Developer sees the failure** in GitHub Actions UI with details
4. **Developer fixes** the issue (upgrade package, patch code, or add exception)
5. **Push again** — pipeline re-runs from scratch

6. What Happens After ECR

Once the image passes all scans and is pushed to ECR:

- **ArgoCD** picks up the new tag from `helmfiles/` and syncs to the cluster
- **Trivy runs again** on the ECR repo (optional — scheduled scan)
- **ECR lifecycle policy** expires old untagged images after 1 day

Summary

Scanner	Target	Fails on
gitleaks	Git commits	Any secret found
npm audit	Node deps	HIGH / CRITICAL CVE
pip-audit	Python deps	HIGH / CRITICAL CVE
Bandit	Python code	Medium+ severity
Semgrep	All code	Rule violations
Trivy	Docker image	HIGH / CRITICAL CVE

Every push gets scanned before it reaches ECR. Nothing deploys without passing all checks.